KOSOVA RADAR — V1 IMPLEMENTATION SPEC

Dokument pune për Claude • Build, test, deploy dhe kontroll i V1

1. Mission for Claude

Build Kosova Radar V1 as a production-ready beta, not a prototype. The goal is to ingest real Kosovo events and deals from a small controlled set of sources, normalize and verify them, expose them through a React frontend, and provide an admin workflow for review and correction.

Do not implement future V2/V3 complexity unless it is required by V1 interfaces. Create clean extension points instead.

2. Non-Negotiable Product Rules

Never publish invented facts. Missing source data stays null/unknown.

Every public item must have a source and freshness timestamp.

Expired/stale items must not appear as active without an explicit state.

AI suggestions are data, not truth; validation and source traceability remain mandatory.

Admin actions are audited.

All external source fetching must be rate-limited and compliant with the source's access rules.

3. Recommended Repository

Create a monorepo or clearly separated solution:

Backend: /src/KosovaRadar.Api, /src/KosovaRadar.Application, /src/KosovaRadar.Domain, /src/KosovaRadar.Infrastructure

Frontend: /src/kosova-radar-web

Tests: /tests/KosovaRadar.UnitTests, /tests/KosovaRadar.IntegrationTests, /tests/KosovaRadar.E2E

Docs: /docs/architecture, /docs/api, /docs/sources

4. Domain Model

User: Id, Email, PasswordHash/Identity fields, Role, Status, CreatedAt.

Source: Id, Name, BaseUrl, SourceType, IsActive, PollIntervalMinutes, LastFetchedAt, LastSuccessAt, LastErrorAt, ReliabilityScore/metadata.

Event: Id, Title, Description, VenueId, CityId, StartsAt, EndsAt, PriceText, PriceAmount, Currency, ImageUrl, SourceId, SourceUrl, VerificationStatus, Confidence, LastVerifiedAt, CreatedAt, UpdatedAt.

Deal: Id, Title, Description, MerchantId/optional, OldPrice, NewPrice, Currency, DiscountPercent, ValidFrom, ValidUntil, SourceId, SourceUrl, VerificationStatus, Confidence, LastVerifiedAt.

Venue, City, Category, Tag, Media, CandidateItem, ReviewAction, AuditLog, JobExecution.

5. Database Rules

Use SQL Server with EF Core migrations.

Use UTC internally; convert to Europe/Pristina only at presentation boundaries.

Indexes: Event StartsAt, CityId, CategoryId, VerificationStatus, SourceId; Deal ValidUntil, SourceId, VerificationStatus.

Unique/near-unique constraints where practical; never rely only on title uniqueness.

Store source URLs and hashes/fingerprints for deduplication.

Soft-delete or explicit status where auditability matters.

6. Source Adapter Contract

Create an interface such as ISourceAdapter with: CanHandle, FetchAsync, ParseAsync, NormalizeAsync.

Each adapter returns normalized candidates, not database entities.

Implement 2–3 concrete adapters first using sources with stable public event/deal pages. Do not build a generic scraper that tries to handle every website.

Add a generic RSS/JSON/HTML adapter only where the structure is reliable.

7. Background Jobs

SourceFetchJob: scheduled fetch per source.

CandidateProcessingJob: parse/normalize/classify.

VerificationJob: validate fields and freshness.

DeduplicationJob: compare candidates against active records.

ExpirationJob: mark expired/stale records.

SourceHealthJob: aggregate failures and alert admin.

All jobs need retry policy, idempotency and structured logs.

8. AI Contract

Create an AI provider abstraction so the model vendor can be changed later.

Prompt the model to output strict JSON matching a schema. Validate with server-side JSON schema/domain validation.

Use AI for: event/deal classification, field extraction from unstructured text, category suggestion, duplicate similarity assistance, concise summaries.

Do not use AI as the final authority for dates, prices or URLs when the source contains contradictory information.

Persist model name/version, request id, confidence and processing timestamp for audit/cost analysis.

9. Candidate → Review → Publish

New item enters Candidate.

Validation produces: Valid, NeedsReview, Rejected.

Admin UI shows source evidence beside extracted fields.

Approve publishes; Reject stores reason; Edit updates normalized data while retaining original source evidence; Merge links duplicates.

Only Published items appear in the public feed.

10. Public API

GET /api/events?city=&from;=&to;=&category;=&search;=&page;=

GET /api/events/{id}

GET /api/deals?...

GET /api/deals/{id}

GET /api/cities

GET /api/categories

Admin endpoints must be separated and role protected.

Use consistent pagination, validation, error envelopes and OpenAPI documentation.

11. Frontend Pages

/ — feed and filters

/events — event listing

/events/:id — event detail

/deals — deal listing

/deals/:id — deal detail

/search — unified search

/admin — dashboard

/admin/review — candidate queue

/admin/sources — source health/configuration

/admin/items/:id — edit/review

12. UX Requirements

Mobile-first responsive UI.

Clear freshness label: e.g. Verified recently / Last checked.

Clear source link.

Expired items visually excluded from active lists.

Empty states, loading states, error states and retry controls.

Use accessible semantic HTML and keyboard navigation for admin.

13. Testing Plan

Unit: domain validation, expiry, discount calculation, normalization, dedupe scoring.

Integration: database migrations, repository queries, source adapter pipeline, API authorization.

E2E: admin login → candidate review → publish → public event → expiry.

Contract tests for each source adapter using saved fixtures so source website changes can be detected.

Load test public feed and search endpoints.

Security test admin authorization and common injection/validation cases.

14. Seed Data & Fixtures

Create realistic Kosovo sample cities, venues, categories and at least 20 fixture candidates. Mark synthetic fixtures clearly so they can never accidentally be treated as live production content.

Include cases: missing time, missing price, duplicate title, changed time, expired event, invalid URL, non-Kosovo location, conflicting source data.

15. CI/CD

On pull request: build backend/frontend, run unit tests, integration tests where infrastructure is available, lint/typecheck frontend, validate migrations.

On staging deploy: run smoke tests and health checks.

Production deployment requires successful CI and database migration review.

Use Docker for repeatable local/staging environments.

16. Observability

Structured logs with correlation/job ids.

Metrics: source fetch success rate, processing latency, AI error rate, candidate-to-publish rate, stale item count, API latency.

Admin-visible source health page.

Do not log secrets or sensitive user data.

17. V1 Acceptance Criteria

AC1: A configured source can be fetched automatically on schedule.

AC2: A valid event can move from source → candidate → review → published.

AC3: A valid deal can move through the same lifecycle.

AC4: Duplicate candidates are detected or surfaced for review.

AC5: Expired events/deals leave active feeds automatically.

AC6: Every published item exposes source and freshness metadata.

AC7: Admin can edit/reject/merge items and actions are audited.

AC8: A source failure does not crash the whole ingestion system.

AC9: AI failure falls back to NeedsReview rather than inventing content.

AC10: Automated tests cover the critical flows and all release gates pass.

18. Implementation Order

Step 1: Repository + solution + Docker + SQL Server + EF Core.

Step 2: Domain entities + migrations + seed.

Step 3: Public API + OpenAPI + validation.

Step 4: React shell + routing + public feed/detail pages.

Step 5: Admin auth + review UI.

Step 6: Source abstraction + first adapter.

Step 7: Background jobs + first end-to-end ingestion.

Step 8: AI extraction/verification abstraction + controlled provider integration.

Step 9: Deduplication + expiry + source health.

Step 10: Second/third source adapters.

Step 11: Automated tests + E2E + fixtures.

Step 12: staging deployment + real-source verification.

Step 13: controlled client beta.

Step 14: fix reliability issues based on real usage; then tag V1.0.

19. Claude Working Rules

Before coding, produce a short architecture summary and task checklist from this document.

Work in small vertical slices; keep the application runnable after each slice.

Do not silently invent requirements. If a decision is needed, document the assumption and choose the least irreversible option.

After each milestone: build, run tests, inspect failures, fix them, and report exactly what changed.

Do not mark a task complete because code compiles; acceptance criteria and tests must pass.

At the end, produce: setup instructions, environment variables, migration commands, source configuration, test commands, known limitations, and a V1 release checklist.

20. Handoff Prompt to Claude

Use this PDF as the authoritative V1 implementation specification for Kosova Radar. Implement the system end-to-end in the repository, test every critical flow, and keep future versions as extension points rather than implementing them prematurely. Start by inspecting the repository and then create the architecture/task breakdown. Do not ask me to manually design every technical detail; make reasonable engineering decisions, document them, and keep the solution production-oriented.